

COMP102 Lecture 13: NoSQL Databases

Jalal Maaouni

June 29, 2026



University
Mohammed VI
Polytechnic

THE UM6P
VANGUARD
CENTER

#Empowering Minds.

Plan

1 Why NoSQL after SQL?

2 What is NoSQL?

3 Documents and JSON

4 Schema-less does not mean careless

5 MongoDB basics

6 SQL vs document stores

7 Recap

WHY NoSQL AFTER SQL?

What SQL gave us

- ▶ clear table structure;
- ▶ primary keys and foreign keys;
- ▶ normalized schemas;
- ▶ joins across related tables;
- ▶ integrity rules enforced by the DBMS.

SQL databases are strong when data is structured, consistent, and relationship-heavy.

But tables are flat

id	name	major
1	Amal	CS
2	Yassine	Math

- ▶ A row is flat.
- ▶ A cell should hold one atomic value.
- ▶ Lists, nested objects, and flexible attributes do not fit naturally.

A more realistic student profile

A student may have:

- ▶ one name;
- ▶ one major;
- ▶ several phone numbers;
- ▶ several skills;
- ▶ an address with city, street, and zip code;
- ▶ preferences that may change over time.

This data is not naturally flat. It has lists and nested structure.

Trying to force nested data into SQL

Bad design:

```
1 CREATE TABLE Student (  
2     id INTEGER PRIMARY KEY,  
3     name TEXT,  
4     major TEXT,  
5     phones TEXT,  
6     skills TEXT,  
7     address TEXT  
8 );
```

This violates the spirit of relational design: lists and structured objects are hidden inside text fields.

Relational fix: more tables

```
1 Student(id, name, major)
2 StudentPhone(student_id, phone)
3 StudentSkill(student_id, skill)
4 StudentAddress(student_id, city, street, zip)
5 StudentPreference(student_id, key, value)
```

This is cleaner than stuffing lists into cells, but the schema becomes heavier.

Where SQL starts to feel awkward

- ▶ Many small tables may be needed for one object-like entity.
- ▶ Reading one student profile may require several joins.
- ▶ Adding a new optional field may require schema changes.
- ▶ Different students may not all have the same attributes.

This does not mean SQL is bad. It means some data shapes are not naturally tabular.

Object-like data in Python

```
1 student = {  
2     "id": 1,  
3     "name": "Amal",  
4     "major": "CS",  
5     "phones": ["0600000000", "0611111111"],  
6     "skills": ["Python", "SQL", "Git"],  
7     "address": {  
8         "city": "Rabat",  
9         "street": "Avenue Mohammed V"  
10    }  
11 }
```

Many applications already manipulate data in this shape.

Some applications read whole objects

Examples:

- ▶ loading a user profile;
- ▶ displaying a product page;
- ▶ reading application settings;
- ▶ storing a blog post with metadata;
- ▶ saving a form submission with optional fields.

If the application usually reads the whole object, storing the whole object together can be convenient.

The motivation for document databases

- ▶ Keep related object-like data together.
- ▶ Store nested fields directly.
- ▶ Allow documents in the same collection to vary.
- ▶ Reduce the need for joins in object-heavy applications.

Document databases are designed around documents, not flat rows.

The right comparison

SQL databases	Document databases
structured tables	JSON-like documents
fixed columns	flexible fields
joins recombine data	embedding keeps data together
strong schema discipline	flexible schema discipline

NoSQL is not automatically better. It solves a different modeling problem.

WHAT IS NoSQL?

NoSQL: the broad idea

Definition

NoSQL refers to a family of database models that do not use the relational table model as their main structure.

- ▶ Not every database stores data as tables.
- ▶ Not every database uses joins as the main way to combine data.
- ▶ Not every database requires a fixed schema before inserting data.

NoSQL does not mean “no structure”. It means a different structure from relational tables.

NoSQL is not one thing

Model	Main idea	Example
Document store	JSON-like documents	MongoDB
Key-value store	key maps directly to value	Redis
Wide-column store	large distributed column families	Cassandra
Graph database	nodes and edges	Neo4j

Today we only focus on document stores.

Key-value stores

key $\longrightarrow$ value

- ▶ The database behaves like a large dictionary.
- ▶ We retrieve a value using its key.
- ▶ Useful for caching, sessions, counters, and fast lookups.

Think of a Python dictionary, but managed by a database system.

Key-value example

```
1 cache = {  
2     "user:1:name": "Amal",  
3     "user:1:theme": "dark",  
4     "course:CS200:title": "Algorithms"  
5 }
```

This model is simple and fast, but it is not designed for complex querying.

Wide-column stores

- ▶ Data is organized into rows and column families.
- ▶ Designed for very large distributed datasets.
- ▶ Common in systems that need high write throughput.
- ▶ Less natural for beginners than tables or documents.

Wide-column stores are important in large-scale systems, but they are not our focus here.

Graph databases

- ▶ Data is stored as nodes and edges.
- ▶ Relationships are first-class data.
- ▶ Useful when the connections are the main object of interest.

Examples:

- ▶ social networks;
- ▶ recommendation graphs;
- ▶ fraud detection networks;
- ▶ dependency graphs.

Graph-style thinking

```
1 (Student)-[:ENROLLED_IN]->(Course)
2 (Student)-[:FRIEND_OF]->(Student)
3 (Course)-[:REQUIRES]->(Course)
```

Graph databases are useful when asking questions about paths, neighborhoods, and relationships.

Document stores

- ▶ Data is stored as documents.
- ▶ Documents look like JSON objects.
- ▶ A document can contain nested fields.
- ▶ A document can contain arrays.
- ▶ Documents in the same collection may have different fields.

This is the NoSQL model we will study in this lecture.

Document example

```
1 {  
2   "id": 1,  
3   "name": "Amal",  
4   "major": "CS",  
5   "skills": ["Python", "SQL", "Git"],  
6   "address": {  
7     "city": "Rabat",  
8     "street": "Avenue Mohammed V"  
9   }  
10 }
```

This shape is closer to a Python dictionary than to a flat SQL row.

Why focus on document stores?

- ▶ They are easy to understand after Python dictionaries.
- ▶ They handle nested data naturally.
- ▶ They are common in web and application backends.
- ▶ MongoDB is the standard introductory example.

Document stores are the clearest contrast with relational tables.

Relational vs document mindset

Relational mindset	Document mindset
split facts into tables	keep related object data together
use foreign keys	use embedded fields or references
use joins to recombine	often read one document directly
schema first	document shape can evolve

Different model, different design questions.

DOCUMENTS AND JSON

Start from a Python dictionary

```
1 student = {  
2     "id": 1,  
3     "name": "Amal",  
4     "major": "CS"  
5 }
```

- ▶ A dictionary stores key-value pairs.
- ▶ Each key names a piece of information.
- ▶ Each value stores the actual data.

From Python dictionary to document

```
1 {  
2     "id": 1,  
3     "name": "Amal",  
4     "major": "CS"  
5 }
```

A document is a structured object made of fields and values.

Vocabulary

Python dictionary	Document database
dictionary	document
key	field name
value	field value
list	array
nested dictionary	embedded document

What is JSON?

Definition

JSON, short for JavaScript Object Notation, is a text format for representing structured data.

```
1 {  
2     "id": 1,  
3     "name": "Amal",  
4     "major": "CS",  
5     "active": true,  
6     "skills": ["Python", "SQL"],  
7     "address": {  
8         "city": "Rabat",  
9         "street": "Avenue Mohammed V"  
10    }  
11 }
```

JSON object

```
1 {  
2     "id": 1,  
3     "name": "Amal",  
4     "major": "CS",  
5     "active": true  
6 }
```

- ▶ JSON objects look like Python dictionaries.
- ▶ Field names are written as strings.
- ▶ Values may be strings, numbers, booleans, arrays, or nested objects.

Python vs JSON detail

Python	JSON
True	true
False	false
None	null
single or double quotes	double quotes

JSON is language-independent. Python dictionaries are Python objects.

Arrays

```
1 {  
2   "id": 1,  
3   "name": "Amal",  
4   "skills": ["Python", "SQL", "Git"]  
5 }
```

- ▶ An array stores several values under one field.
- ▶ This is natural in a document.
- ▶ In a normalized SQL design, this usually becomes another table.

Nested objects

```
1 {  
2   "id": 1,  
3   "name": "Amal",  
4   "address": {  
5     "city": "Rabat",  
6     "street": "Avenue Mohammed V",  
7     "zip": "10000"  
8   }  
9 }
```

A nested object stores structured data inside another document.

A fuller student document

```
1 {  
2   "id": 1,  
3   "name": "Amal",  
4   "major": "CS",  
5   "phones": ["0600000000", "0611111111"],  
6   "skills": ["Python", "SQL", "Git"],  
7   "address": {  
8     "city": "Rabat",  
9     "street": "Avenue Mohammed V"  
10  }  
11 }
```

Why this shape matters

- ▶ The student profile is stored as one object.
- ▶ Lists are represented directly.
- ▶ Nested information is represented directly.
- ▶ The application can read the whole profile at once.

Document databases are built for this object-like shape.

Collection = group of documents

```
1 students = [  
2     {  
3         "id": 1,  
4         "name": "Amal",  
5         "major": "CS"  
6     },  
7     {  
8         "id": 2,  
9         "name": "Yassine",  
10        "major": "Math"  
11    }  
12 ]
```

A collection is a group of related documents.

Relational table vs document collection

Relational database	Document database
table	collection
row	document
column	field
cell value	field value

A collection is similar to a table only at a high level. Documents are not forced to be flat rows.

Documents in the same collection may vary

```
1 students = [  
2     {  
3         "id": 1,  
4         "name": "Amal",  
5         "major": "CS",  
6         "skills": ["Python", "SQL"]  
7     },  
8     {  
9         "id": 2,  
10        "name": "Yassine",  
11        "major": "Math"  
12    }  
13 ]
```

Both are student documents, but they do not have exactly the same fields.

Accessing document fields in Python

```
1 student = {  
2     "id": 1,  
3     "name": "Amal",  
4     "major": "CS",  
5     "skills": ["Python", "SQL"]  
6 }  
7  
8 print(student["name"])  
9 print(student["skills"][0])
```

Expected output:

```
1 Amal  
2 Python
```


Accessing nested fields in Python

```
1 student = {  
2     "name": "Amal",  
3     "address": {  
4         "city": "Rabat",  
5         "street": "Avenue Mohammed V"  
6     }  
7 }  
8  
9 print(student["address"]["city"])
```

Expected output:

```
1 Rabat
```

Section recap

- ▶ A document is a JSON-like object.
- ▶ A field is a named value inside a document.
- ▶ Documents can contain arrays.
- ▶ Documents can contain nested objects.
- ▶ A collection is a group of related documents.

Document stores feel natural after Python dictionaries and lists.

SCHEMA-LESS DOES NOT MEAN CARELESS

What schema-less means

Definition

Schema-less means the database does not force every document in a collection to have exactly the same fields.

- ▶ One document may have extra fields.
- ▶ Another document may omit optional fields.
- ▶ Nested structures may evolve over time.

Documents can vary

```
1 {  
2   "id": 1,  
3   "name": "Amal",  
4   "major": "CS",  
5   "skills": ["Python", "SQL"]  
6 }
```

```
1 {  
2   "id": 2,  
3   "name": "Yassine",  
4   "major": "Math"  
5 }
```

Both documents represent students, but only one has a `skills` field.

Why flexibility helps

- ▶ Applications change over time.
- ▶ New optional fields can be added gradually.
- ▶ Older documents can still exist without those fields.
- ▶ Different users may naturally have different data.

Schema flexibility is useful when the shape of the data evolves.

Adding a new field later

Old document:

```
1 {  
2   "id": 1,  
3   "name": "Amal",  
4   "major": "CS"  
5 }
```

New document:

```
1 {  
2   "id": 3,  
3   "name": "Salma",  
4   "major": "CS",  
5   "scholarship": true  
6 }
```

The collection can contain both shapes during the transition.

But flexibility creates risks

- ▶ Field names may become inconsistent.
- ▶ Types may differ across documents.
- ▶ Required information may be missing.
- ▶ Queries may silently miss some documents.

The database may accept messy data, but the application still has to survive it.

Bad inconsistency: field names

```
1 {  
2   "name": "Amal",  
3   "major": "CS"  
4 }
```

```
1 {  
2   "full_name": "Yassine",  
3   "department": "Math"  
4 }
```

A query for `name` will not find documents that use `full_name` instead.

Bad inconsistency: value types

```
1 {  
2   "name": "Amal",  
3   "year": 2  
4 }
```

```
1 {  
2   "name": "Yassine",  
3   "year": "second"  
4 }
```

The same field should not randomly switch between numbers and strings.

Bad inconsistency: arrays vs strings

```
1 {  
2   "name": "Amal",  
3   "skills": ["Python", "SQL"]  
4 }
```

```
1 {  
2   "name": "Yassine",  
3   "skills": "Python, SQL"  
4 }
```

Both store skills, but only one stores them as a real array.

Schema-less still needs discipline

Good document design still requires decisions:

- ▶ Which fields are expected?
- ▶ Which fields are optional?
- ▶ What type should each field have?
- ▶ Which nested structures are allowed?
- ▶ How should missing values be handled?

Schema-less means flexible schema, not no schema thinking.

Where the schema moves

Relational database	Document database
Schema is declared strongly in tables	Schema is often enforced by application code
Invalid rows are rejected early	Invalid documents may enter if code is careless
Changes require migrations	Shape can evolve more gradually

The responsibility does not disappear. It shifts.

Careful document design

```
1 {  
2   "id": 1,  
3   "name": "Amal",  
4   "major": "CS",  
5   "skills": ["Python", "SQL"],  
6   "address": {  
7     "city": "Rabat",  
8     "street": "Avenue Mohammed V"  
9   }  
10 }
```

- ▶ consistent field names;
- ▶ predictable field types;
- ▶ clear nested structure;
- ▶ optional fields used deliberately.

Rule of thumb

Use flexibility for real variation, not for accidental disorder.

- ▶ Missing optional field: acceptable.
- ▶ Same concept under three different names: bad.
- ▶ Array stored sometimes as text: bad.
- ▶ New field added for a new feature: acceptable.

Your turn

For each case, decide whether it is reasonable flexibility or careless inconsistency.

1. Some students have a `skills` field; others do not.
2. Some documents use `name`; others use `full_name`.
3. Some students have an `address` object; others do not.
4. Some documents store `year` as `2`; others store it as `"second"`.

Schema-less design still needs clear expectations.

MONGODB BASICS

From document design to MongoDB

- ▶ We now understand the document model.
- ▶ Documents are JSON-like objects.
- ▶ Collections group related documents.
- ▶ Schema-less design gives flexibility, but still requires discipline.

MongoDB is one database system that implements this document-store model.

What MongoDB adds

MongoDB gives us a real database around documents:

- ▶ stores documents persistently;
- ▶ organizes documents into collections;
- ▶ assigns each document a unique `_id`;
- ▶ supports queries using JSON-like filters;
- ▶ supports insert, find, update, and delete operations.

The data shape is still document-based. MongoDB provides the engine around it.

The shift in mindset

Before MongoDB	With MongoDB
Python dictionary	document stored in the database
List of dictionaries	collection of documents
Manual filtering in Python	database query with a filter
Manual updates to objects	database update command

MongoDB lets us store and query dictionary-like objects without first flattening them into tables.

MongoDB vocabulary

Relational database	MongoDB
database	database
table	collection
row	document
column	field
primary key	_id

MongoDB stores documents inside collections.

A MongoDB document

```
1 {  
2   "_id": 1,  
3   "name": "Amal",  
4   "major": "CS",  
5   "skills": ["Python", "SQL"],  
6   "address": {  
7     "city": "Rabat",  
8     "street": "Avenue Mohammed V"  
9   }  
10 }
```

- ▶ `_id` uniquely identifies the document.
- ▶ Fields store values.
- ▶ Values may be nested objects or arrays.

Python setup with pymongo

```
1 from pymongo import MongoClient
2
3 client = MongoClient("mongodb://localhost:27017")
4
5 db = client["university"]
6 students = db["students"]
```

This connects Python to a MongoDB database named `university` and a collection named `students`.

Insert one document

```
1 students.insert_one({  
2     "_id": 1,  
3     "name": "Amal",  
4     "major": "CS",  
5     "skills": ["Python", "SQL"]  
6 })
```

`insert_one` adds one document to the collection.

Insert several documents

```
1 students.insert_many([
2     {
3         "_id": 2,
4         "name": "Yassine",
5         "major": "Math"
6     },
7     {
8         "_id": 3,
9         "name": "Salma",
10        "major": "CS",
11        "skills": ["Python", "MongoDB"]
12    }
13 ])
```

Documents in the same collection may have different fields.

Find all documents

```
1 for student in students.find():  
2     print(student)
```

`find()` returns documents from the collection.

Find with a filter

```
1 for student in students.find({"major": "CS"}):  
2     print(student)
```

MongoDB filters are written as JSON-like dictionaries.

Find one document

```
1 student = students.find_one({"_id": 1})  
2  
3 print(student)
```

`find_one` returns one matching document, or `None` if nothing matches.

Query an array field

```
1 for student in students.find({"skills": "Python"}):  
2     print(student["name"])
```

When a field is an array, MongoDB can match documents where the array contains the value.

Nested-field query

```
1 for student in students.find({"address.city": "Rabat"}):  
2     print(student["name"])
```

Dot notation such as `address.city` accesses a field inside a nested document.

Update one document

```
1 students.update_one(  
2     {"_id": 1},  
3     {"$set": {"major": "AI"}}  
4 )
```

- ▶ First dictionary: which document to update.
- ▶ Second dictionary: what change to apply.
- ▶ \$set changes or creates a field.

Add a new field

```
1 students.update_one(  
2     {"_id": 1},  
3     {"$set": {"scholarship": True}}  
4 )
```

In a document database, adding a new field to one document is straightforward.

Update a nested field

```
1 students.update_one(  
2     {"_id": 1},  
3     {"$set": {"address.city": "Casablanca"}}  
4 )
```

Dot notation can also update nested fields.

Delete one document

```
1 students.delete_one({"_id": 2})
```

`delete_one` removes one matching document from the collection.

Delete several documents

```
1 students.delete_many({"major": "CS"})
```

`delete_many` removes all documents matching the filter. Use it carefully.

CRUD recap

Operation	MongoDB method
Create	<code>insert_one</code> , <code>insert_many</code>
Read	<code>find</code> , <code>find_one</code>
Update	<code>update_one</code> , <code>update_many</code>
Delete	<code>delete_one</code> , <code>delete_many</code>

These examples are only the basic interface. The important idea is the document model.

SQL VS DOCUMENT STORES

SQL is better when

- ▶ data has a stable structure;
- ▶ relationships are important;
- ▶ integrity rules must be enforced strongly;
- ▶ many queries combine data from several entities;
- ▶ duplicate facts would create serious problems.

Use SQL when correctness, relationships, and structured querying matter most.

Document stores are better when

- ▶ data is naturally object-like;
- ▶ documents contain nested fields or arrays;
- ▶ different records may have different optional fields;
- ▶ the application often reads or writes the whole object;
- ▶ the data shape changes frequently.

Use document stores when flexibility and object-shaped data matter most.

Example: better fit for SQL

University enrollment:

- ▶ students take many courses;
- ▶ courses contain many students;
- ▶ grades belong to the relationship;
- ▶ course information is shared by many students;
- ▶ duplicated course titles would create update problems.

This is naturally relational: Student, Course, Enrollment.

Example: better fit for documents

Student profile:

- ▶ name and major;
- ▶ optional preferences;
- ▶ list of phone numbers;
- ▶ list of skills;
- ▶ nested address;
- ▶ settings that may evolve over time.

This is naturally document-shaped: one profile can be stored as one document.

Main tradeoff

SQL databases	Document stores
strong schema	flexible schema
normalized data	embedded data
joins are natural	whole-document reads are natural
integrity enforced by DBMS	more responsibility in application code
less duplication by design	possible duplication for convenience

The dangerous mistake

Do not choose NoSQL because it sounds newer. Do not choose SQL because it sounds safer.

Choose based on:

- ▶ the shape of the data;
- ▶ the queries the application needs;
- ▶ the importance of integrity constraints;
- ▶ how often the structure changes;
- ▶ whether duplication is acceptable.

Rule of thumb

Situation	Likely fit
many-to-many relationships	SQL
strict consistency rules	SQL
nested user profiles	Document store
flexible settings or metadata	Document store
heavy reporting across many entities	SQL
read/write one whole object often	Document store

Final comparison

- ▶ SQL asks: how do we separate facts cleanly?
- ▶ Document stores ask: what object does the application usually need?
- ▶ SQL avoids redundancy through normalization.
- ▶ Document stores often reduce joins through embedding.
- ▶ Both require careful design.

The database model should follow the problem, not fashion.

RECAP

What is a document store?

- ▶ A document store saves data as JSON-like documents.
- ▶ A document can contain fields, arrays, and nested objects.
- ▶ The document is often close to the object used by the application.

Document stores organize data around objects, not flat rows.

Collections replace tables

SQL database	Document store
table	collection
row	document
column	field
primary key	_id

A collection groups related documents, but documents are not forced to be identical flat rows.

Flexible fields

- ▶ Documents in the same collection may have different fields.
- ▶ This helps when the application evolves.
- ▶ But inconsistent field names and types create bugs.

Schema-less means flexible schema, not careless data.

The main tradeoff

- ▶ Embedding is convenient but may duplicate facts.
- ▶ Referencing avoids duplication but requires more lookup work.
- ▶ SQL normalizes by default.
- ▶ Document stores often denormalize deliberately.

There is no free design. Every model moves the cost somewhere.

NoSQL is a different fit

- ▶ NoSQL is not automatically better than SQL.
- ▶ SQL is strong for structured data, relationships, and integrity.
- ▶ Document stores are strong for flexible, nested, object-like data.
- ▶ The right choice depends on the data shape and query patterns.

Choose the database model that fits the problem, not the trend.

What you should remember

- ▶ Document stores use JSON-like documents.
- ▶ Collections group documents.
- ▶ Documents may have flexible fields.
- ▶ MongoDB is a common document database.
- ▶ Filters are written as JSON-like dictionaries.
- ▶ Flexibility requires discipline.